

TP4 -Mettre en œuvre une stack

d'Observabilité moderne avec Prometheus & Grafana

Environnement :

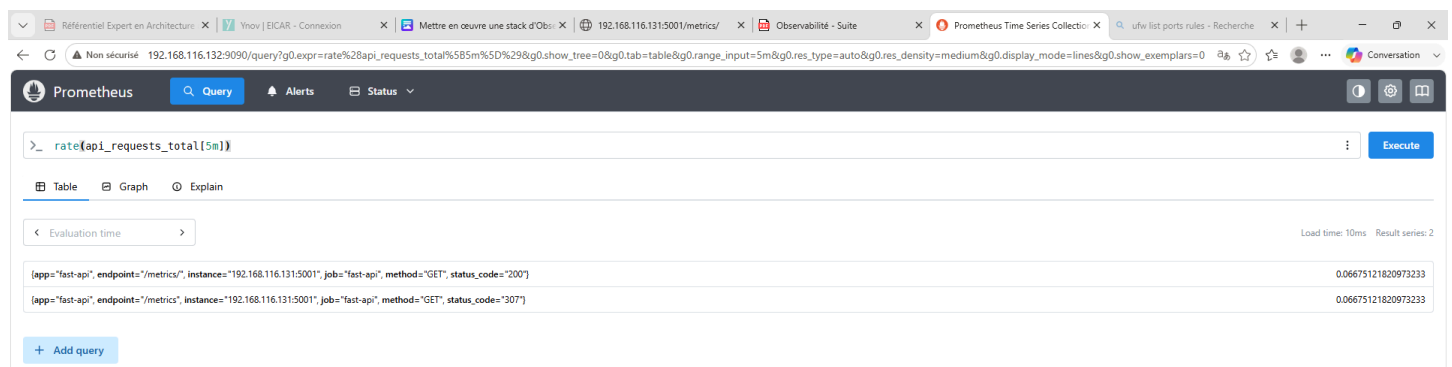
- VMWare (Debian 13 x2)
- VisualCode
- Répertoire de travail :

→ HOME\Documents\Mastere_2\Supervision\TP4 - Obsevailite Avancee

Collecte Prometheus et Visualisation Grafana : Exercices PromQL

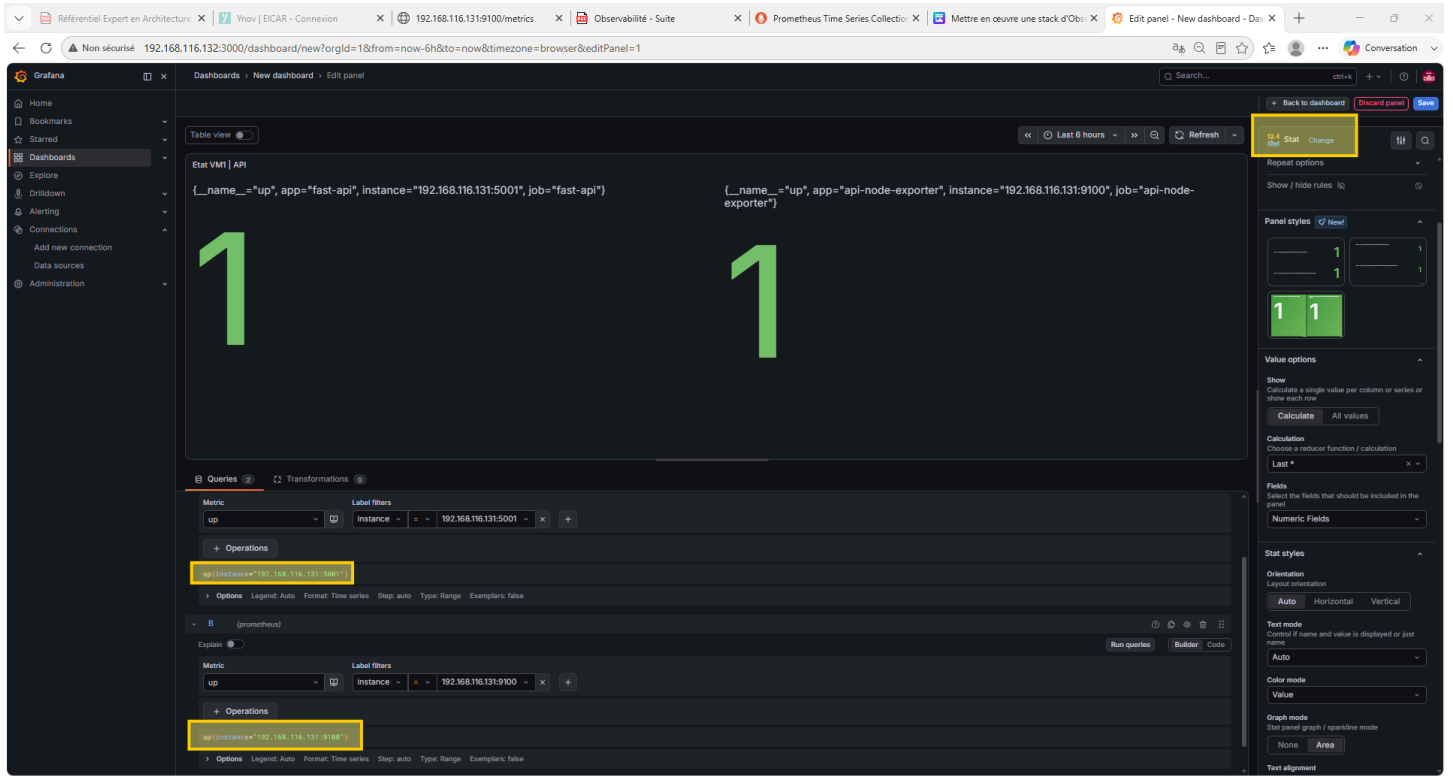
Trouvez les requêtes PromQL permettant d'extraire :

- Le taux moyen de requêtes par seconde sur l'API lors des 5 dernières minutes.

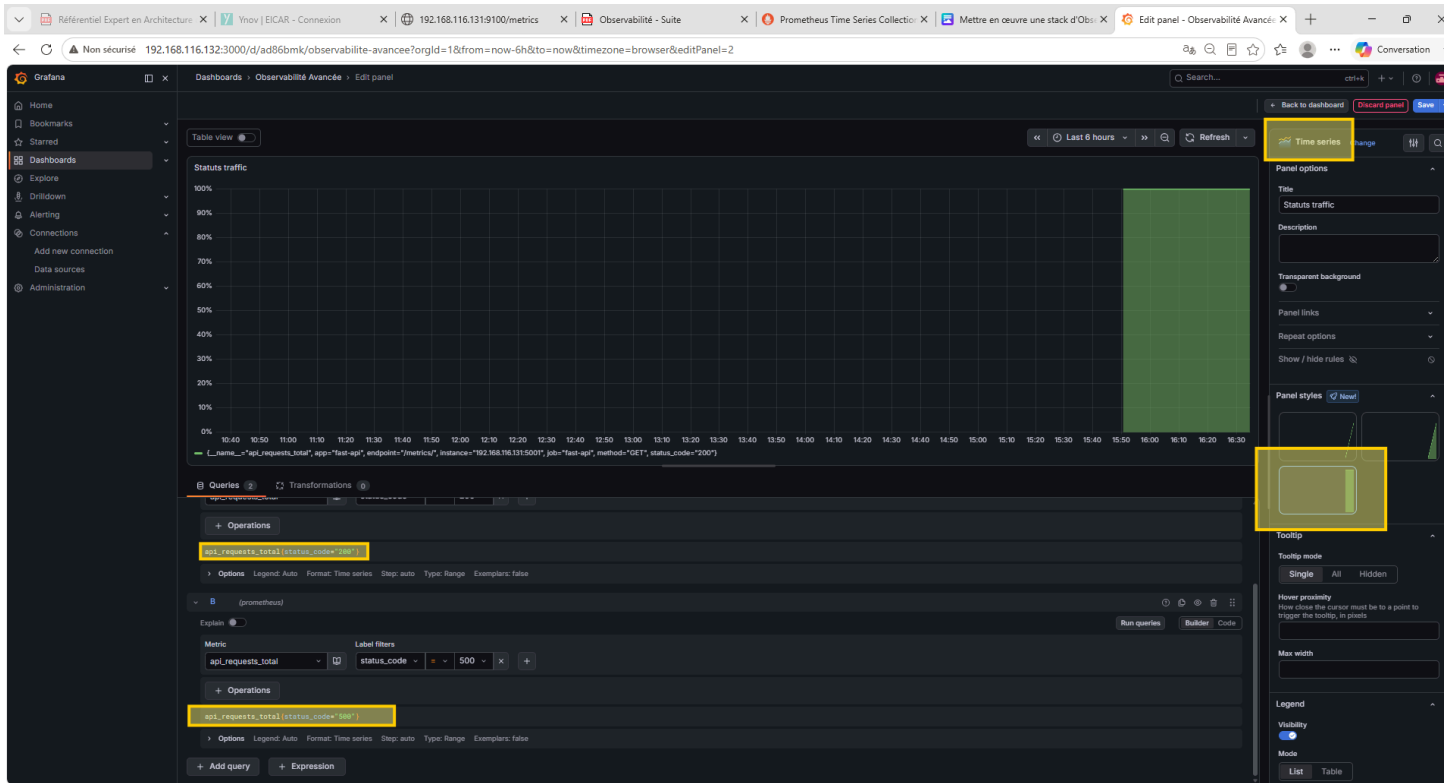


Taux moyen de 0.06675121820973233 requêtes par secondes, tout statut compris.

- Le pourcentage de requêtes en erreur HTTP 500 par rapport au volume total.



- Un graphique temporel empilé affichant le trafic par code de retour (200 vs 500).



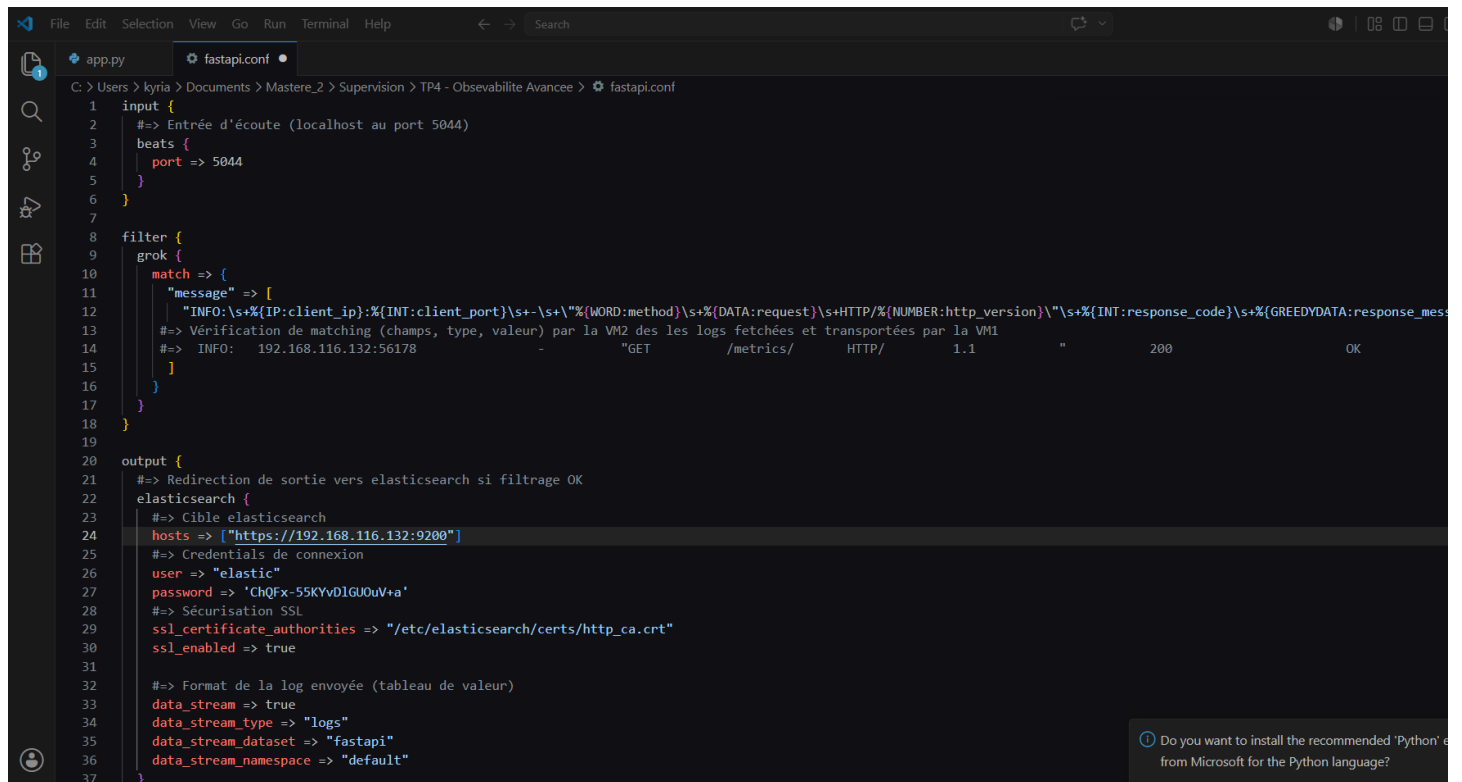
- Un graphique de distribution des latences basé sur votre histogramme.

Analyse :

- Analyse de performance : "Faire un tir de charge sur l'API FastAPI (avec wrk). Observer l'évolution du graphique de latence sur Grafana. À quel percentile commencez-vous à voir une dégradation des performances ? Pourquoi ?"

?

- Explication de texte : Documenter, dans le rapport, chaque ligne du filtre Grok fourni par l'IA.



```
1 input {
2   #=> Entrée d'écoute (localhost au port 5044)
3   beats {
4     port => 5044
5   }
6 }
7
8 filter {
9   grok {
10    match => {
11      "message" => [
12        "INFO:\s+%(IP:client_ip):%(INT:client_port)\s+-\s+\"%(WORD:method)\s+\"%(DATA:request)\s+HTTP/%(NUMBER:http_version)\" \s+\"%(INT:response_code)\s+\"%(GREEDYDATA:response_mes
13      #=> Vérification de matching (champs, type, valeur) par la VM2 des les logs fetchées et transportées par la VM1
14      #=> INFO: 192.168.116.132:56178 - "GET /metrics/ HTTP/ 1.1 " 200 OK
15    ]
16  }
17 }
18 }
19
20 output {
21   #=> Redirection de sortie vers elasticsearch si filtrage OK
22   elasticsearch {
23     #=> Cible elasticsearch
24     hosts => ["https://192.168.116.132:9200"]
25     #=> Credentials de connexion
26     user => "elastic"
27     password => 'ChQFv-55KYvDIGUOuV+a'
28     #=> Sécurisation SSL
29     ssl_certificate_authorities => "/etc/elasticsearch/certs/http_ca.crt"
30     ssl_enabled => true
31
32     #=> Format de la log envoyée (tableau de valeur)
33     data_stream => true
34     data_stream_type => "logs"
35     data_stream_dataset => "fastapi"
36     data_stream_namespace => "default"
37   }
38 }
```

- La panne réseau : "Bloquer un port spécifique sur le pare-feu de la VM 1. Que se passe-t-il sur Prometheus ? Que devient la valeur de la métrique up ? Combien de temps met l'alerte pour passer de Pending à Firing ? quel impact sur Kibana ?"

1) Désactivation du port d'exposition de l'API :

```
root@FastApi:~# ufw deny 5001
Rule updated
Rule updated (v6)
root@FastApi:~#
```

Entre le moment où l'alerte passe de **Pending** à **Firing**, il s'écoule 3 minutes, comme configuré dans l'alerte associée. **Pending** indique ici que la cible répond aux conditions d'alerte. Seulement, il faut s'assurer qu'elle y répond selon le délai imposé avant d'alerter.

Firing indique qu'une alerte a été lancée :

The screenshot shows the Prometheus Alerts page. The alert is titled 'fastapi-alerts' and is in a 'FIRING' state. The alert rule is 'up{instance="192.168.116.131:5001"} == 0'. The alert is triggered by a 'FastAPI' application. The alert description is 'La machine FastAPI n'est plus joignable depuis au moins 3 minutes.' and the summary is 'Instance FastAPI KO ?'. The alert labels are 'alertname="API KO"', 'app="fast api"', 'instance="192.168.116.131:5001"', 'job="fast api"', and 'severity="Haute"'. The alert is active since '5m 21.783s' and has a value of '0'.

L'impact sur Kibana ? L'API ne renvoie pas de log car étant donné que son port bloqué, aucun accès aux URL n'est possible, donc filebeat n'a plus de logs à récupérer, ce qui fait qu'à la fin plus aucune logs n'est reçue sur Kibana :

The screenshot shows the Elastic Kibana dashboards page. The dashboard is titled 'Repartition - Code API (FastAPI)'. The dashboard is empty, showing 'No results found' for both the left and right panels. The dashboard is created using KQL syntax and is set to 'Last 3 minutes'.

- La panne de disque / saturation : "Générez une boucle infinie qui sature la RAM de la VM 1. À partir de quel seuil l'alerte système se déclenche-t-elle ?"

?

Réfléchir aux questions suivantes :

- Si je lance une requête qui met 3 secondes à répondre, quelle ligne de votre code calcule cette durée et où est-elle stockée dans Prometheus ?"

```

@app.middleware("http")
async def prometheus_middleware(request: Request, call_next):
    start_time = time.time()

    response = await call_next(request)

    duration = time.time() - start_time
    endpoint = request.uri.path
    method = request.method
    status_code = response.status_code

    # Mise à jour des métriques
    api_requests_total.labels(method, endpoint, status_code).inc()
    api_request_duration_seconds.labels(method, endpoint).observe(duration)

    return response

```

- Quelle est la différence fondamentale entre la manière dont Prometheus récupère les données (Pull) et la manière dont Filebeat/Logstash les envoient (Push) ? Pourquoi avoir choisi ces deux modèles différents pour les métriques et les logs ?

Prometheus **tire** les données à travers des URL /metrics des serveurs surveillés, pour les analyser.

Filebeat **pousse** les données en destination d'un serveur de supervision logstash qui analyse et traite les métriques reçues.

La différence fondamentale se trouve au niveau de la manière dont travaillent les agents. Pour Prometheus, ce sont les agents qui mettent à disposition les métriques pour que le nœud maître puisse les chercher, tandis que pour logstash, c'est le nœud maître qui attend que les agents viennent lui apporter les métriques.

- Si Elasticsearch crash, que deviennent les logs envoyés par Filebeat pendant la panne ? Sont-ils perdus ?

Elles sont stockées dans une file d'attente et les reprocess une fois la panne terminée.